Documentație Tehnică

Forum Application

Vancea A.-Urzică T.- Popa D.

Software Design

1 aprilie 2026

Cuprins

1 Introducere **3**
1.1 Obiectivele documentației . . . 3

2 Stiva de Tehnologii **3**
2.1 Configurarea `pom.xml` . . . 4
2.2 Configurarea aplicației (`application.yaml`) . . . 4

3 Arhitectura Aplicației **4**
3.1 Structura pachetelor . . . 5
3.2 Descrierea straturilor . . . 6
3.2.1 Controller Layer . . . 6
3.2.2 Service Layer . . . 6
3.2.3 Repository Layer . . . 6

4 Modelul de Date **6**
4.1 Entitățile principale . . . 6
4.1.1 Answer . . . 6
4.1.2 Attachment (clasă abstractă) . . . 7
4.1.3 AnswerDTO . . . 7
4.1.4 AnswerRepository . . . 8

5 Diagrame de Clase **8**
5.1 Diagrama entităților (Entity Layer) . . . 8
5.2 Diagrama stratului de servicii (Service Layer) . . . 9

6 Endpoint-uri REST API **9**
6.1 UserController — `/api/users` . . . 9
6.2 QuestionController — `/api/questions` . . . 9
6.3 AnswerController — `/api/answers` . . . 10

7 Principiile SOLID — Analiză cu Exemple din Cod **10**
7.1 S — Single Responsibility Principle (SRP) . . . 10
7.2 O — Open/Closed Principle (OCP) . . . 11
7.3 L — Liskov Substitution Principle (LSP) . . . 11
7.4 I — Interface Segregation Principle (ISP) . . . 12
7.5 D — Dependency Inversion Principle (DIP) . . . 12

8 Sumar — Respectarea Principiilor SOLID **13**

9 Concluzii **13**

1 Introducere

Forum App este o aplicație web de tip forum universitar, dezvoltată utilizând framework-ul **Spring Boot 4.0**. Aplicația permite utilizatorilor să creeze conturi, să publice întrebări, să ofere răspunsuri și să organizeze conținutul prin intermediul etichetelor (tags). Imaginile pot fi atașate atât la întrebări, cât și la răspunsuri.

1.1 Obiectivele documentației

- Prezentarea **arhitecturii** aplicației (stratificată, pe trei niveluri)
- Descrierea **stivei de tehnologii** utilizate
- Analiza respectării **principiilor SOLID** cu exemple concrete din codul sursă
- Prezentarea structurii bazei de date și a relațiilor dintre entități prin **diagrame de clase**

2 Stiva de Tehnologii

Categorie	**Tehnologie**	**Descriere**
Limbaj	Java 21	Versiunea LTS cu suport pentru virtual threads, pattern matching, sealed classes
Framework	Spring Boot 4.0	Framework de aplicație enterprise cu auto-configurare și container IoC
Web	Spring WebMVC	Implementare REST API cu adnotări `@RestController`, `@RequestMapping`
Persistență	Spring Data JPA	Abstractizare peste Hibernate ORM; interfețe `CrudRepository` pentru operații CRUD
ORM	Hibernate	Mapare obiect-relațională automată cu suport pentru relații complexe
Bază de date	MySQL	SGBD relațional; conexiune prin `mysql-connector-j`
Validare	Spring Validation	Validare declarativă prin adnotări Jakarta Bean Validation
Generare cod	Lombok	Elimină codul boilerplate: `@Data`, `@Builder`, `@NoArgsConstructor`
Build	Maven	Gestionare dependențe și build cu `pom.xml`

2.1 Configurarea `pom.xml`

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
```

Listing 1: Dependențele principale din pom.xml

2.2 Configurarea aplicației (`application.yaml`)

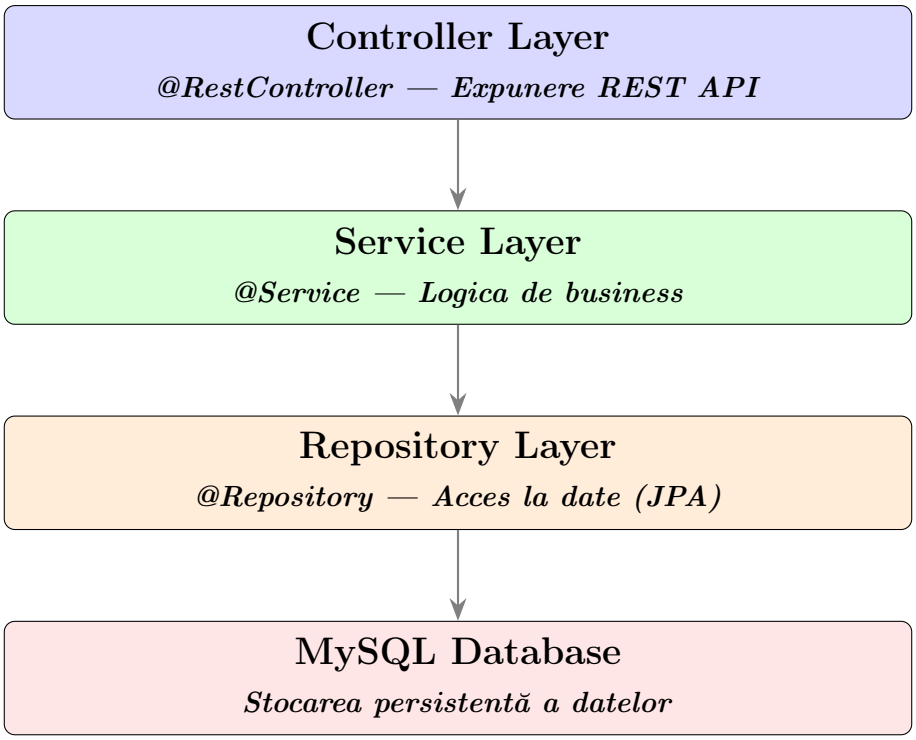
```yaml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/forum-app
         ?createDatabaseIfNotExist=true
    username: root
    password: root
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: create-drop
    show-sql: true
    open-in-view: false
```

Listing 2: Configurarea surselor de date

Notă: Proprietatea `ddl-auto: create-drop` recreează schema bazei de date la fiecare pornire — utilă în faza de dezvoltare.

3 Arhitectura Aplicației

Aplicația urmează o **arhitectură stratificată (Layered Architecture)** pe trei niveluri, conformă cu convențiile Spring Boot:



3.1 Structura pachetelor

```
com.university.forum_app/
  |-- controller/        # REST Controllers
  |      |-- UserController.java
  |      |-- QuestionController.java
  |      |-- AnswerController.java
  |-- dto/               # Data Transfer Objects
  |      |-- UserDTO.java
  |      |-- QuestionDTO.java
  |      |-- AnswerDTO.java
  |      |-- TagDTO.java
  |-- entity/            # Entitati JPA
  |      |-- User.java
  |      |-- Question.java
  |      |-- Answer.java
  |      |-- Tag.java
  |      |-- Attachment.java    (clasa abstracta)
  |      |-- QuestionImage.java
  |      |-- AnswerImage.java
  |      |-- Role.java          (enum)
  |      |-- Status.java        (enum)
  |-- repository/        # Interfete Spring Data
  |      |-- UserRepository.java
  |      |-- QuestionRepository.java
  |      |-- AnswerRepository.java
  |      |-- TagRepository.java
  |-- service/           # Logica de business
  |      |-- UserService.java
```

```
|      |-- QuestionService.java
|      |-- AnswerService.java
|      |-- TagService.java
|-- Application.java   # Punct de intrare
```

3.2 Descrierea straturilor

3.2.1 Controller Layer

Stratul de prezentare expune endpoint-urile REST API:

- Primirea cererilor HTTP (`GET`, `POST`, `PUT`, `DELETE`)
- Deserializarea corpului cererii (`@RequestBody`, `@RequestPart`)
- Delegarea logicii către stratul `Service`
- Returnarea răspunsurilor cu coduri HTTP corespunzătoare

3.2.2 Service Layer

Stratul de logică de business:

- Validări de business (username unic, titlu unic, etc.)
- Conversia între entități și DTO-uri (`mapEntityToDTO`, `mapDTOToEntity`)
- Gestionarea tranzacțiilor (`@Transactional`)
- Gestionarea fișierelor (upload imagini pe disc via `MultipartFile`)

3.2.3 Repository Layer

Stratul de acces la date:

- Extinde `CrudRepository<Entity, Long>` pentru operații CRUD standard
- Definește metode de interogare derivate din numele metodei (query derivation)
- Spring generează automat implementarea la runtime

4 Modelul de Date

4.1 Entitățile principale

4.1.1 Answer

```java
@Entity
@Table(name = "answers")
@Data @Builder @AllArgsConstructor @NoArgsConstructor
public class Answer {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```java
    @Column(name = "answer_id")
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User author;

    @Column(name = "text")
    private String text;

    @CreationTimestamp
    @Column(name = "date_and_time", updatable = false)
    private LocalDateTime date;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "question_id", nullable = false)
    private Question question;

    @OneToMany(mappedBy = "answer", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<AnswerImage> images;
}
```

Listing 3: Entitatea Answer

4.1.2 Attachment (clasă abstractă)

```java
@MappedSuperclass
@Data @NoArgsConstructor @AllArgsConstructor @SuperBuilder
public abstract class Attachment {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String imageUrl;
}
```

Listing 4: Clasa abstractă Attachment — moștenire JPA

QuestionImage și **AnswerImage** extind `Attachment`, moștenind `id` și `imageUrl`, adăugând doar relația `@ManyToOne` specifică.

4.1.3 AnswerDTO

```java
@Data @Builder @AllArgsConstructor @NoArgsConstructor
public class AnswerDTO {
    private Long answerId;
    private Long questionId;
    private Long userId;
    private LocalDateTime dateTime;
    private List<String> imageUrls;
}
```

Listing 5: AnswerDTO — câmpuri de transfer

4.1.4 AnswerRepository

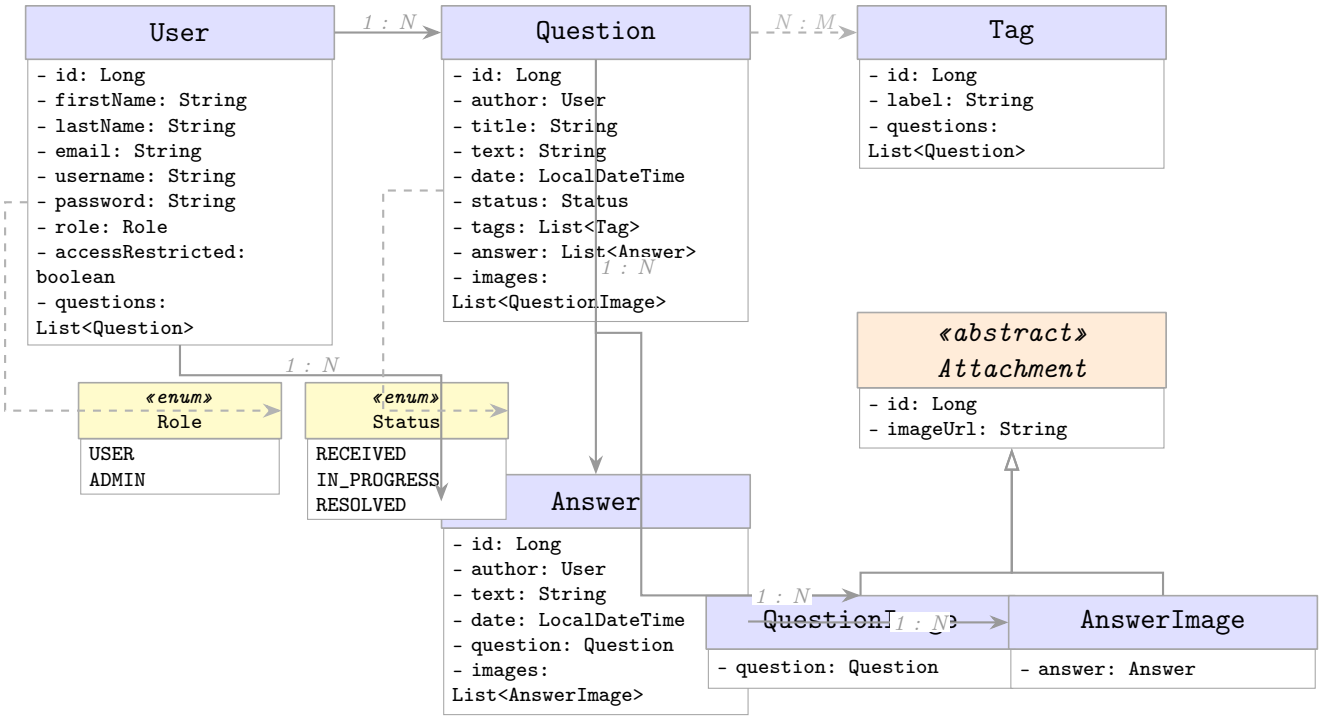
```
@Repository
public interface AnswerRepository extends CrudRepository<Answer, Long> {
    List<Answer> findByQuestionId(Long questionId);
    List<Answer> findByAuthorId(Long userId);
    Answer findAnswerById(Long answerId);
}
```

Listing 6: AnswerRepository — metode de interogare

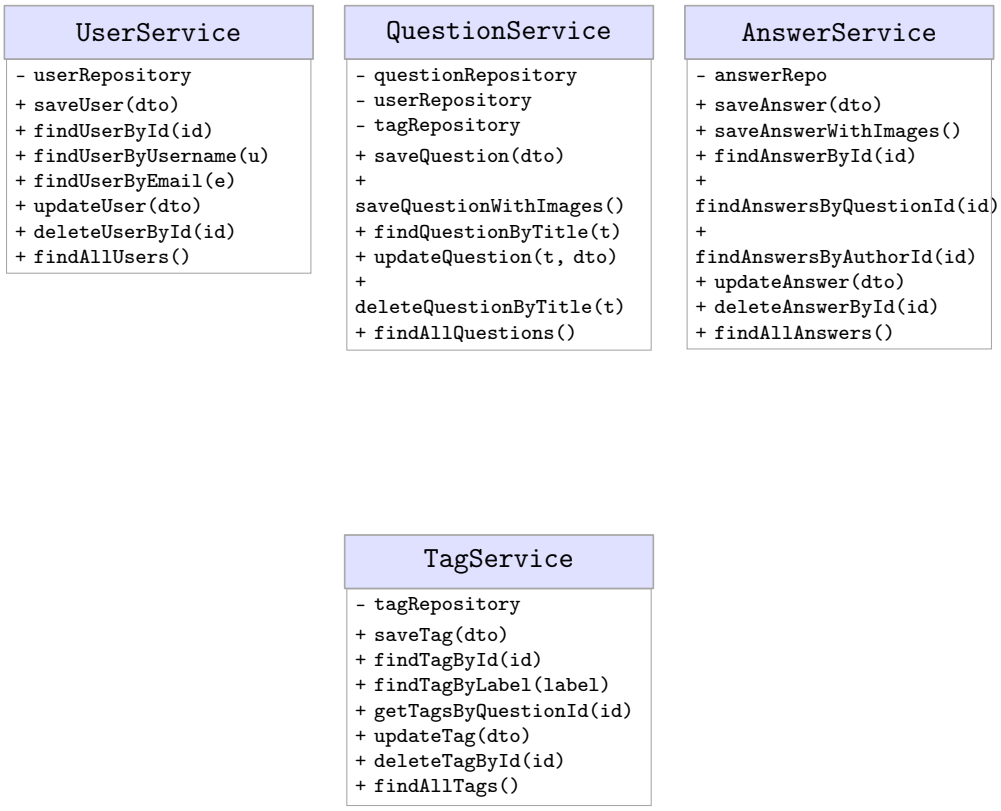
5 Diagrame de Clase

5.1 Diagrama entităților (Entity Layer)

Diagrama de mai jos prezintă structura claselor din pachetul `entity`, cu atributele principale, tipul de date și relațiile JPA dintre ele.



5.2 Diagrama stratului de servicii (Service Layer)



6 Endpoint-uri REST API

6.1 UserController — `/api/users`

HTTP	Endpoint	Descriere
POST	`/api/users`	Creare utilizator nou
GET	`/api/users`	Lista tuturor utilizatorilor
GET	`/api/users/{id}`	Obținere utilizator după ID
GET	`/api/users/username/{username}`	Căutare după username
GET	`/api/users/email/{email}`	Căutare după email
PUT	`/api/users/{id}`	Actualizare utilizator
DELETE	`/api/users/{id}`	Ștergere utilizator

6.2 QuestionController — `/api/questions`

HTTP	Endpoint	Descriere
POST	`/api/questions`	Creare întrebare nouă
POST	`/api/questions/with-images`	Creare întrebare cu imagini
GET	`/api/questions`	Lista tuturor întrebărilor
GET	`/api/questions/title/{title}`	Căutare după titlu

PUT	`/api/questions/title/{title}`	Actualizare întrebare
DELETE	`/api/questions/title/{title}`	Ștergere întrebare

6.3 AnswerController — `/api/answers`

HTTP	Endpoint	Descriere
POST	`/api/answers`	Creare răspuns nou
POST	`/api/answers/with-images`	Creare răspuns cu imagini
GET	`/api/answers`	Lista tuturor răspunsurilor
GET	`/api/answers/{id}`	Obținere răspuns după ID
GET	`/api/answers/question/{questionId}`	Răspunsuri per întrebare
GET	`/api/answers/user/{userId}`	Răspunsuri per utilizator
PUT	`/api/answers/{id}`	Actualizare răspuns
DELETE	`/api/answers/{id}`	Ștergere răspuns

7 Principiile SOLID — Analiză cu Exemple din Cod

Principiile SOLID promovează codul mentenabil, extensibil și flexibil. Fiecare principiu este ilustrat cu un exemplu concret din codul proiectului.

7.1 S — Single Responsibility Principle (SRP)

> *"O clasă trebuie să aibă un singur motiv pentru a se schimba."*

Separarea Controller — Service: `AnswerController` se ocupă **exclusiv** de comunicarea HTTP, delegând întreaga logică către `AnswerService`:

```java
@RestController
@RequestMapping("/api/answers")
public class AnswerController {
    @Autowired
    private AnswerService answerService;  // singura dependenta

    @PostMapping
    public ResponseEntity<?> createAnswer(@RequestBody AnswerDTO answerDTO) {
        try {
            AnswerDTO savedAnswer = answerService.saveAnswer(answerDTO);
            return new ResponseEntity<>(savedAnswer, HttpStatus.CREATED);
        } catch (IllegalArgumentException e) {
            return new ResponseEntity<>(e.getMessage(), HttpStatus.BAD_REQUEST);
        }
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<?> deleteAnswer(@PathVariable Long id) {
        try {
            answerService.deleteAnswerById(id);
            return new ResponseEntity<>(
```

```
                "Answer with id " + id + " was deleted successfully.",
                HttpStatus.OK);
        } catch (IllegalArgumentException e) {
            return new ResponseEntity<>(e.getMessage(), HttpStatus.
                NOT_FOUND);
        }
    }
}
```

Listing 7: SRP — AnswerController delegează logica

Validarea și persistența sunt responsabilitatea exclusivă a lui `AnswerService`, nu a controller-ului.

7.2 O — Open/Closed Principle (OCP)

> *"Entitățile software trebuie să fie deschise pentru extensie, dar închise pentru modificare."*

Ierarhia Attachment: Adăugarea unui nou tip de atașament nu necesită modificarea clasei `Attachment` — se creează o nouă subclasă:

```
// Clasa de baza -- inchisa pentru modificare
@MappedSuperclass @SuperBuilder
public abstract class Attachment {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String imageUrl;
}

// Extensie 1 -- fara sa modificam Attachment
@Entity @Table(name = "question_images") @SuperBuilder
public class QuestionImage extends Attachment {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "question_id")
    private Question question;
}

// Extensie 2 -- fara sa modificam Attachment
@Entity @Table(name = "answer_images") @SuperBuilder
public class AnswerImage extends Attachment {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "answer_id")
    private Answer answer;
}
```

Listing 8: OCP — QuestionImage și AnswerImage extind fără modificare

Un eventual `MessageImage` ar necesita doar `class MessageImage extends Attachment` — fără a atinge codul existent.

7.3 L — Liskov Substitution Principle (LSP)

> *"Obiectele unei superclase trebuie să poată fi înlocuite cu obiecte ale subclaselor lor fără a afecta corectitudinea programului."*

Utilizarea metodei moștenite `getImageUrl()`: Atât `QuestionImage` cât și `AnswerImage` sunt folosite identic prin metoda moștenită din `Attachment`:

```
// In QuestionService -- subtip QuestionImage
.imageUrls(question.getImages() != null
    ? question.getImages().stream()
        .map(QuestionImage::getImageUrl).toList()
    : new ArrayList<>())

// In AnswerService -- subtip AnswerImage, acelasi comportament
.imageUrls(answer.getImages() != null
    ? answer.getImages().stream()
        .map(AnswerImage::getImageUrl).toList()
    : new ArrayList<String>())
```

Listing 9: LSP — getImageUrl() functional identic in ambele subtipuri

Substituirea este transparentă: `getImageUrl()` este moștenită din `Attachment` și funcționează corect în ambele subclase, fără modificarea logicii apelante.

7.4 I — Interface Segregation Principle (ISP)

"Clienții nu trebuie forțați să depindă de interfețe pe care nu le utilizează."

Repository-uri specifice per entitate: Fiecare repository expune doar metodele relevante domeniului său:

```
@Repository
public interface AnswerRepository extends CrudRepository<Answer, Long> {
    List<Answer> findByQuestionId(Long questionId);
    List<Answer> findByAuthorId(Long userId);
    Answer findAnswerById(Long answerId);
}

@Repository
public interface TagRepository extends CrudRepository<Tag, Long> {
    Tag findTagByLabel(String tagLabel);
    List<Tag> findTagByQuestionsId(Long questionId);
}
```

Listing 10: ISP — AnswerRepository cu metode specifice domeniului

`AnswerRepository` nu conține metode `findByTitle` sau `findByLabel` — fiecare interfață este minimală și coerentă cu domeniul său.

7.5 D — Dependency Inversion Principle (DIP)

"Modulele de nivel înalt nu trebuie să depindă de modulele de nivel scăzut. Ambele trebuie să depindă de abstracțiuni."

Injecția de dependențe și depedența pe interfețe: `AnswerService` depinde de interfața `AnswerRepository`, nu de o implementare concretă. Containerul IoC Spring injectează automat implementarea la runtime:

```
@Service
public class AnswerService {
    @Autowired
```

```
    private AnswerRepository answerRepo; // INTERFATA, nu implementare concreta

    @Value("${app.upload.dir.answers:uploads/answers}")
    private String uploadDir;  // configuratie externalizata, nu hardcoded

    // Spring Data genereaza implementarea la runtime
    // AnswerService nu cunoaste detaliile SQL/JDBC
}
```

Listing 11: DIP — AnswerService depinde de interfata si primeste configuratie externa

Similar, `AnswerController` depinde exclusiv de `AnswerService` și nu are nicio referință la repository sau la baza de date.

8 Sumar — Respectarea Principiilor SOLID

	Principiu	**Cum este respectat**
S	Single Responsibility	Separare clară: Controller (HTTP), Service (business logic), Repository (date), DTO (transfer), Entity (model)
O	Open/Closed	Ierarhia `Attachment` → `QuestionImage`/`AnswerImage`; extensie `CrudRepository` fără modificare framework
L	Liskov Substitution	`QuestionImage` și `AnswerImage` substituibile ca `Attachment`; `getImageUrl()` funcționează identic în ambele
I	Interface Segregation	Repository-uri specifice per entitate; DTO-uri specializate per domeniu
D	Dependency Inversion	`@Autowired` pe interfețe; Controller → Service → Repository (interfețe); `@Value` pentru configurație

9 Concluzii

Proiectul **Forum App** demonstrează aplicarea principiilor de software design într-un context practic, utilizând ecosistemul Spring Boot. Principalele puncte forte ale arhitecturii sunt:

1. **Separarea clară a responsabilităților** prin arhitectură stratificată pe 3 niveluri
2. **Utilizarea pattern-ului DTO** pentru izolarea stratului de prezentare de cel de persistență
3. **Extensibilitate** prin ierarhia `Attachment` și interfețele `CrudRepository`
4. **Injecția de dependențe** gestionată complet de containerul Spring IoC

5. **Gestionarea tranzacțiilor** declarativă prin adnotarea `@Transactional`
6. **Suport pentru upload de imagini** cu organizare pe disc și servire prin URL-uri publice

Stiva tehnologică aleasă (Java 21, Spring Boot 4, Hibernate, MySQL, Lombok, Maven) asigură un fundament robust pentru extinderea viitoare — autentificare (Spring Security), notificări în timp real sau integrare cu un frontend modern.